

Data structures and Algorithms

Pointers



Introduction to Pointers. Why Pointers?

Let us consider we have a variable `abc` in our program: `Int abc=10;`
How the system knows where **abc** is stored?

num_1	var	int
num_2	var	int
num_5	var	int

We have a **symbol table**. A **Symbol table** is a data structure used by the compiler and is generated at compilation time where each identifier in program's source code is stored along with information associated with it relating to its declaration.

The symbol table stores many information(name of variable, memory address, the data type, scope).

The symbol table will store the variable **abc** and its address (ex: 800) and when I print variable **abc** stored in the address of 800 and print the value stored there.

```
cout<<&abc;  
c;
```

Now I want to print the address of the variable **abc** using address of operator **&** which will print hexadecimal memory address of **abc**.



```
0x7ffff605b6f4
```

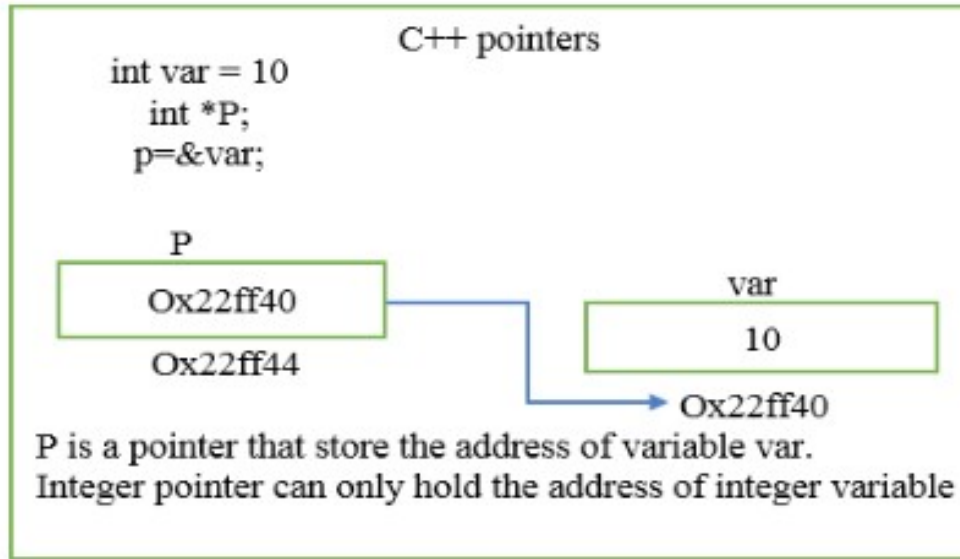
Question: How can we store the address of **abc**: int, char, and float can't store the address of `abc`.

That's where comes Pointers

1 Definition of Pointers

A **pointer** is a variable which contains the **address** in memory of another variable. There can be a pointer to any variable type. The unary or monadic operator & gives the **“address of a variable”**. The indirection or dereference operator * gives the **“contents of an object pointed to by a pointer”**.

C++ gives the power to manipulate the data in the computer's memory directly. The space in the memory can be assigned and de-assigned. This is done using pointer variables. Pointer variables are variables that points to a specific address in the memory pointed to by another variable.



Interpretation of pointer



2 Dereference operator (*)

The asterisk sign (*) in the cout statements is called the dereference operator. If the dereference operator is used, the “value pointed by” a pointer is got.

The statement:

cout << *ptr; means to print the content of the memory space pointed by ptr.

Example: use a pointer in calculating area of circle

```
#include<iostream>
using namespace std;
int main ()
{
float PI = 3.14;
float *PIptr;
float r=5, A;
float x;
PIptr=&PI; //PIptr contains the address of PI
x=*PIptr; // value stored in PI(3.14)is assigned to x.
A=r*r*x;
cout<<"The area of circle is"<<A<<endl
return 0;
}
```



Dereference operator *

The Dereference operator goes to memory location and print the value of variable.

```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int i=10;
5      int *p=&i;
6      //Printing the address of the variable
7      cout<<"The address of i:"<<&i<<endl;
8      cout<<"The address of i using pointer:"<<p<<endl;
9      cout<<"The address of p:"<<&p<<endl;
10     // Printing the value of the variable
11     cout<<"The value of i is:"<<i<<endl;
12     cout<<"The value of i using Pointer:"<<*p<<endl;
13     //Let us change the value of i
14     i=20;
15     cout<<"The value of i is:"<<i<<endl;
16     cout<<"The value of i using Pointer:"<<*p<<endl;
17     //We can change the value of i using pointer
18     *p=60;
19     cout<<"The value of i is:"<<i<<endl;
20     cout<<"The value of i using Pointer:"<<*p<<endl;
21     return 0;
22 }
```

Let us consider the address of i as 500 and address of p=600.

i=20 means go to the address of i(500) and change the value to 20.

*p=60 means go to symbol table and search for p with memory address 600 and see the value stored there which is 500 and go to 500 address and change the value stored there to 60

```
The address of i:0x7ffefce9718c
The address of i using pointer:0x7ffefce9718c
The value of i is:10
The value of i using Pointer:10
The value of i is:20
The value of i using Pointer:20
The value of i is:60
The value of i using Pointer:60
```



3 Reference operator

The ampersand sign (&) is called the reference operator. If the reference operator is used, it will get the “address of” a variable. The `ptr = &x;` means that it stores the address of the variable `x` in the pointer `ptr`.

Example:

```
#include<iostream>
using namespace std;
int main()
{
    int x;
    int *ptr;
    x = 5;
    ptr = &x;//
    cout << *ptr;
    getchar();
    return 0;
}
```

Note: If you forget to place `*` (in front of the pointer) in the `cout` statement, you will print the address of integer `x`.



4 Declaring pointer variables

A pointer is declared by using the * (asterisk) operator before an identifier. It is necessary to initialize the pointer before you can use a pointer in for instance a cout statement.

Due to the ability of a pointer to directly refer to the value that it points

to, it becomes necessary to specify in its declaration which data type a pointer is going to point to. It is not the same thing to point to a char as to point to an int or a float.

Pointer declaration Syntax:

`data_type *pt_name;`

Where data_type is the data type of the value that the pointer is intended to point to.

The following are three declarations of pointers and all of them are pointers and will occupy the same amount of space in memory:

```
int *number;  
char * character;  
float * greatnumber;
```

//some Pointer declaration examples

```
int *iptr;  
// iptr is pointer to int  
char *cptr; // cptr is a pointer to char  
float *fptr;  
// fptr is a pointer to float
```



Special considerations for Character variable

The address of char is being treated as a nul-terminated string and is displaying the contents of that address. If you cast the pointers to void *, you will get the results you desire.

Example:

```
char b='c';
```

```
cout<<(void*)&b<<endl;
```



The size of pointers is the same

The size of pointer is fixed as it store the address of a variable. Either a pointer to char or pointer to integers the sizeof their pointers will be the same and this will vary from compiler to compiler and system to system.

```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int i=20;
5      cout<<&i<<endl;
6      int *p=&i;
7      cout<<p<<endl;
8      cout<<*p<<endl;
9      char c='d';
10     char *p2=&c;
11     // The size of i and c are the same
12     cout<<"The size of i:"<<sizeof(i)<<endl;
13     cout<<"The size of c:"<<sizeof(c)<<endl;
14     // The size of all pointers is the same.
15     cout<<"The size of p:"<<sizeof(p)<<endl;
16     cout<<"The size of p2:"<<sizeof(p2)<<endl;
17     return 0;
18 }
19
```



```
0x7fff7b196b34
0x7fff7b196b34
20
The size of i:4
The size of c:1
The size of p:8
The size of p2:8
```

5 Initialization of Pointer Variable

Pointers can be initialized to point to specific locations at the very moment they are defined, You need to initialize a pointer by assigning it a valid address.

pointer_variable = &variable;

Example of initialization:

```
int *ptr; // pointer to int declared, value
          undefined
int x = 5; // int declared and initialized to 5
cout << x; //prints 5
cout << *ptr; //Error! Prints undefined value,
              since ptr not initialized
ptr = &x; //ptr now contain value of x
cout << *ptr; // Prints 5
```

```
#include<iostream>
using namespace std;
int main ()
{
// Declaring Variables and Pointer
int Num=10;
int *Ptr;
Ptr=&Num;
// Printing Values of Pointer
cout <<"\n Value Of Num :" << Num << endl ;
cout <<"\n Address Of Num :" << &Num << endl;
cout <<"\n Value Of Ptr :" << Ptr << endl;
cout <<"\n Address Of Ptr :" << &Ptr << endl;
cout <<"\n Ptr's Pointer Value:" << *Ptr << endl;
cout <<"\n Ptr Equal to &Num :" << *(&Num) << endl;
// Change Values of Using Pointer
cout <<"\n\n We Can Change Value Of Num,Without
Using Num" << endl;
*Ptr=100;
cout <<"\n Address Of Num :"<<&Num << endl;
cout <<"\n Ptr's Pointer Value:"<<*Ptr << endl;
getchar();
}
```



Pointer initialisation

Never dereference any garbage address, instead initialise your pointer to null

```
#include<iostream>
using namespace std;
int main () {
    int a=20;
    int *p=&a; // Valid
    int *q=NULL;
    q=p; // Valid
    cout<<p<<endl;
    cout<<q<<endl;// Print same address as p
    cout<<*q<<endl;// Print 20
    cout<<*p<<endl;// Print 20
    return 0;}
```

```
#include<iostream>
using namespace std;

int main(){
    int j;
    cout<<j<<endl;
    int *p = NULL;
    *p = 50;
    return 0;
}
```



6 Graphical representation of pointers for referencing

Memory allocation.

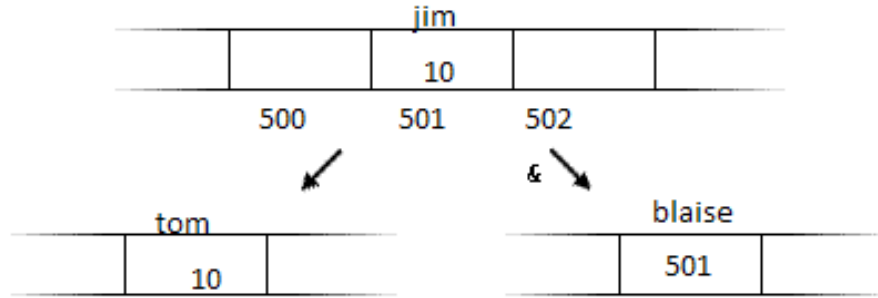
Consider the following,

```
int jim,tom,*blaise;
```

```
jim = 10;
```

```
tom= jim;
```

```
blaise = &jim;
```



First, we have assigned the value 10 to jim (a variable whose address in memory we have assumed to be 501).

The second statement copied to tom the content of variable jim which is 10.

The third statement copies to blaise not the value contained in jim but a reference to it (its address, which we have assumed to be 501).

The following is graphical representation of the value contained in each variable after the execution of this, are shown in the following diagram:



8. Incrementing a pointer

Incrementing pointer variable depends upon data type of the Pointer variable. It increments its value by the number of bytes of its data type

Incrementing a pointer is generally used in array because we have contiguous memory in array and we know the contents of next memory location.

This can be done `ptr++` or `+ptr`

```
1  #include<iostream>
2  using namespace std;
3  const int MAX = 3;
4  int main () {
5      int var[MAX] = {10, 100, 200};
6      int *ptr;
7      // let us have array address in pointer.
8      ptr = var;
9      for (int i =0 ; i < MAX; i++) {
10         cout << "Address of var["<< i << "] = ";
11         cout << ptr << endl;
12         cout << "Value of var[" << i << "] = ";
13         cout << *ptr << endl;|
14         // point to the next location
15         ptr++;
16     }
17     getchar();
18     return 0;}
```

```
Address of var[0] = 0x7fffcccbafadc
Value of var[0] = 10
Address of var[1] = 0x7fffcccbafae0
Value of var[1] = 100
Address of var[2] = 0x7fffcccbafae4
Value of var[2] = 200
```



Note: Difference incrementing the pointer and the value we are pointing to:

Note: You should note that `(*ptr)++` increments the value the pointer is pointing to and not the address while `*ptr++` increments the pointer because the `++` operator has the priority than `*`.

The following:

`*ptr++`
is equivalent to
`*ptr;`
`ptr++;`



```
#include<iostream>
using namespace std;
int main(){
    int a = 50;
    int *ptr = &a;
    cout<<(*ptr)++<<endl;
    cout<<ptr<<endl;
    cout<<a<<endl;
    cout<<(*ptr)++<<endl;
    cout<<ptr<<endl;
    cout<<*ptr<<endl;
    cout<<a<<endl;
    return 0;
}
```



9. Decrement a pointer

The following example shows the program decrementing a pointer, which decreases its value by the number of bytes of its data type

```
1  #include<iostream>
2  using namespace std;
3  const int MAX = 3;
4  int main () {
5      int var[MAX] = {10, 100, 200};
6      int *ptr;
7      // let us have address of the last element in pointer.
8      ptr = &var[MAX-1];
9      for (int i =MAX ; i>0; i--) {
10         cout << "Address of var[" << i << "] = ";
11         cout << ptr << endl;
12         cout << "Value of var[" << i << "] = ";
13         cout << *ptr << endl;
14         // point to the previous location
15         ptr--;
16     }
17     getchar();
18     return 0;
19 }
```

```
Address of var[3] = 0x7fffbb09bbd4
Value of var[3] = 200
Address of var[2] = 0x7fffbb09bbd0
Value of var[2] = 100
Address of var[1] = 0x7fffbb09bbcc
Value of var[1] = 10
```



10 Arrays and pointers

The name of an array is a pointer to the first element in the array.

```
int a[10];  
int *aptr = a;  
aptr = a ;// is equivalent to aptr =  
&a[0];  
aptr+5;// is equivalent to &a[5]  
*(aptr+5); // is equivalent to a[5]
```

```
1  #include <iostream>  
2  using namespace std;  
3  int main()  
4  {  
5      float arr[10];  
6      float *ptr;  
7      cout << "Displaying address using arrays: " << endl;  
8      for (int i = 0; i < 10; ++i)  
9      {  
10         cout << "&arr[" << i << "] = " << &arr[i] << endl;  
11     }  
12     // ptr = &arr[0]  
13     ptr = arr;  
14     cout<<"\n Displaying address using pointers: "<< endl;  
15     for (int i = 0; i < 10; ++i)  
16     { cout << "ptr + " << i << " = "<< ptr + i << endl;  
17     }  
18     getchar();  
19     return 0;  
20 }
```

```
Displaying address using arrays:  
&arr[0] = 0x7ffcdab8dce0  
&arr[1] = 0x7ffcdab8dce4  
&arr[2] = 0x7ffcdab8dce8  
&arr[3] = 0x7ffcdab8dcec  
&arr[4] = 0x7ffcdab8dcf0  
&arr[5] = 0x7ffcdab8dcf4  
&arr[6] = 0x7ffcdab8dcf8  
&arr[7] = 0x7ffcdab8dcfc  
&arr[8] = 0x7ffcdab8dd00  
&arr[9] = 0x7ffcdab8dd04
```

```
Displaying address using pointers:  
ptr + 0 = 0x7ffcdab8dce0  
ptr + 1 = 0x7ffcdab8dce4  
ptr + 2 = 0x7ffcdab8dce8  
ptr + 3 = 0x7ffcdab8dcec  
ptr + 4 = 0x7ffcdab8dcf0  
ptr + 5 = 0x7ffcdab8dcf4  
ptr + 6 = 0x7ffcdab8dcf8  
ptr + 7 = 0x7ffcdab8dcfc  
ptr + 8 = 0x7ffcdab8dd00
```

Array vs Pointer: Program to print the sum of array

```
#include <iostream>
using namespace std;
int main()
{
    int sum,i;
    int a[5]={10,20,30,40,50};
    for (sum = *a, i = 1; i < 5; i++)
sum+= *(a + i);
cout<<sum<<endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main()
{
    int sum,i;
    int a[5]={10,20,30,40,50};
    int *p;
    p=a;
    for (sum = *a, p = a+1; p < a+5; p++)
sum += *p;

    cout<<sum<<endl;
    return 0;
}
```



11 Comparison of pointers

Pointers may be compared by using relational operators, such as ==, <, and >.

The following program incrementing the variable pointer so long as the address to which it points is either less than or equal to the address of the last element of the array, which is &var[MAX - 1].

```
1  #include<iostream>
2  using namespace std;
3  int MAX=3;
4  int main () {
5      int var[MAX] = {10, 100, 200};
6      int *ptr;
7      // let us have address of the first element in pointer
8      ptr = var;
9      int i = 0;
10     while ( ptr <= &var[MAX - 1] ) {
11         cout << "Address of var[" << i << "] = ";
12         cout << ptr << endl;
13         cout << "Value of var[" << i << "] = ";
14         cout << *ptr << endl;
15         // point to the previous location
16         ptr++;
17         i++;
18     }
19     getchar();
20     return 0;}
21
```

```
Address of var[0] = 0x7ffcaed54030
Value of var[0] = 10
Address of var[1] = 0x7ffcaed54034
Value of var[1] = 100
Address of var[2] = 0x7ffcaed54038
Value of var[2] = 200
```



Consider the following codes, answer the Questions below:

1) What will happen to these codes?

```
int a = 100, b = 200;  
int *p = &a, *q = &b;  
p = q;
```

- a) b is assigned to a
- b) p points to b
- c) a is assigned to b
- d) q now point to a

2) What will be the output of the block of codes?

```
int a = 15;  
int b = 20;  
int *c = &b;  
*c = 15;  
cout << a << " " << b << endl;
```

- a) 15 20
- b) 20 15
- c) 15 15
- d) 20 20



Pointer test/10 Marks

3. Which of the following gives the memory address of integer variable a?

- a) *a;
- b) a;
- c) &a;
- d) address(a);

4) What will be the output ?

```
float score = 15.5;
float p = 7.5;
float *ptr = &score;
(*ptr)++;
*ptr = p;
cout << *ptr << " " << score << " " << p;
```

- a) 7.5 15.5 7.5
- b) 7.5 16.5 7.5
- c) 7.5 7.5 7.5
- d) 16.5 16.5 7.5

5. What will be the output ?

```
int a = 20;
int b = 30;
int *c = &b;
a = *c;
*c = *c + 1;
cout << a << " " << b << endl;
```

- a) 31 31
- b) 20 31
- c) 31 20
- d) 30 31

6. Which of the following gives the memory address of variable 'b' pointed by pointer 'a' i.e.

```
int b = 10;
int *a = &b;
```

- a) a;
- b) *a;
- c) &a;
- d) address(a);



Study the codes below and provide the correct answer.

7) Which is correct?

```
int k = 20;  
int *ptr = &k;  
int *q = ptr;  
(*q)++;  
cout << k << endl;
```

- a) 20
- b) 21
- c) Error
- d) None of these

8) What is the output?

```
int k = 60;  
int *ptr = &k;  
cout << (*ptr)++ << " ";  
cout << k << endl;
```

- a) 60 61
- b) 61 60
- c) 60 60
- d) 61 61

9) Which one is correct?

```
int *ptr = 0;  
int r = 30;  
*ptr = r;  
cout << *ptr << endl;
```

- a) 30
- b) 0
- c) Error
- d) None of these



What is the output?

```
10. #include<iostream>
using namespace std;
int main(){
    int a = 7;
    int b = 17;
    int *c = &b;
    *c = 7;
    cout<<"a="<<a<<" and b= "<<b<<endl;
    c = &a;
    *c = 17;
    cout<<"Then a="<<a<<" and b="<<b<<endl;
    cout<<*c<<endl;
    return 0;
```

```
11. #include<iostream>
using namespace std;
int main(){
    int a = 50;
    int *ptr = &a;
    cout<<(*ptr)++<<endl;
    cout<<a<<endl;
    int *q=ptr;
    (*q)++;
    cout<<(*ptr)++<<endl;
    cout<<*q<<endl;
    cout<<a<<endl;
    return 0;
}
```



Pointers and Reference variable

Consider the following:

```
int n = 5, *p = &n, &r = n;
```

Variable **p** is declared as being of type **int***, a pointer to an integer, and **r** is of type **int&**, an integer reference variable. A reference variable must be initialized in its declaration as a reference to a particular variable, and this reference cannot be changed. This means that a reference variable cannot be null. A reference variable **r** can be considered a different name for a variable **n** so that if **n** changes then **r** changes as well. This is because a reference variable is implemented as a **constant pointer** to the variable. After the three declarations, the printing statement

```
cout << n << ' ' << *p << ' ' << r << endl;
```

outputs 5 5 5. After the assignment

```
n = 7;
```

The same printing statement outputs 7 7 7. Also, an assignment

```
*p = 9;
```

 gives the output 9 9 9, and the assignment

```
r = 10;
```

leads to the output 10 10 10. These statements indicate that in terms of notation, what we can accomplish with dereferencing of pointer variables is accomplished without dereferencing of reference variables. This is no accident because, as mentioned, reference variables are implemented as constant pointers. Instead of the declaration.



Pointers and Reference variable(cont...)

Instead of the declaration

```
int& r = n;
```

we can use a declaration

```
int *const r = &n;
```

where **r** is a constant pointer to an integer, which means that the assignment

```
r = q;
```

where q is another pointer, is an error because the value of r cannot change. However, the assignment

```
*r = 1;
```

is acceptable if n is not a constant integer. It is important to note the difference between the type **int *const** and the type **const int ***. The latter is a type of pointer to a constant integer:

```
const int *s = &m;
```

where m is an integer (whether constant or not) is admissible, but the assignment

```
*s = 2;
```

